How do you run CS223 labs on a Mac without Vivado?

19 and 20 September 2026 · M2, 8 GB, macOS 15 PDF the five project files section 1 prerequisites, 2 the one-line install, 3 the setup by hand, 4 to 8 your lab and the limits, 9 the tested versions

Apple Silicon Mac (M1 or later), a Digilent Basys3, free tools. Everything here was run on an M2 with 8 GB of RAM and macOS 15 on 19 and 20 September 2026. Intel Macs are untested and the installer refuses them.

Two ways in. **Take the one line.** Build by hand only if you want to see each tool go in, or the installer refuses your machine.

	one line	**by hand**
do	section 1, then 2, then 5 to 8	section 1, then 3, then 4 to 8
time after section 1	one command; 3 min 37 s to 4 min 17 s on the tested machine plus about 4 minutes for yosys-slang, 1.4 GB downloaded, about 0.3 GB kept (237 MB measured on 7 October; the optional agent SDK adds 51 MB)	6 steps; about 8 minutes of compiling plus 1.76 GB of full clones, call it 25 minutes
you get	the `dewfpga` command	the same tools plus a Makefile you own
tool versions	three pinned commits, Homebrew versions in section 9	the same three commits, typed by you

Section 4 is the Makefile project the by-hand path uses. Sections 5 to 8 are for both: your own lab, VS Code, the course files that break, what this chain cannot do.

If a step fails. Read the last lines in the terminal. Every error hit during this setup is filed under its exact text, with the fix: nosey-dewdrop.github.io/dewfpga/errors. Not there? Open an issue at github.com/nosey-dewdrop/dewfpga/issues with the last ten lines and the section number.

No Mac or no board with you? The browser simulator runs the same three files (`design.sv`, `tb.sv`, the `.xdc`) with nothing installed: a virtual Basys3, and a testbench view that draws the waveform.

Contents

- 1. What do you need before you start?
- 2. How do you install it in one line?
- 3. How do you build the chain by hand?
- 4. What is in the project folder?
- 5. How do you use it for your own lab?

- 6. How do you set up VS Code?
- 7. Which errors does the CLI stop with, and what do they mean?
- 8. What can it not do?
- 9. Which versions were tested?

---

1. What do you need before you start?

The board. A Digilent Basys3 and its micro-USB cable. Apple Silicon MacBooks have only USB-C ports, so you also need a USB-C to USB-A adapter or hub. The board is powered from the cable; leave the JP2 jumper on USB.

The terminal. Press Cmd Space, type `Terminal`, press Enter. Every command in this guide goes into that window: paste it, press Enter, wait until the line ending in `%` comes back before you paste the next one. Some commands span several lines and end in `\`; copy the whole grey block at once.

Homebrew. It installs developer tools by name, and on a Mac that does not have them it first installs Apple's command line tools (`git`, `make`, the C compiler), which the chain is built with; that is the installer's own `should_install_command_line_tools` step. So this one line covers both:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/ins
tall.sh)"
```

It asks for your Mac password; the letters do not show while you type. At the end it prints a few lines under "Next steps". Paste those into the terminal too: they put `/opt/homebrew/bin` on your PATH, and without them the next command answers `command not found`. Open a new terminal window and check:

```
brew --version
# Homebrew 4.x
```

If `dewfpga install` later stops with `Xcode Command Line Tools missing` (a Homebrew copied over by Migration Assistant, or a tools download that failed), run `xcode-select --install`, click Install in the window, and run it again. If it stops with `Homebrew is installed (...) but not on your PATH`, the "Next steps" lines were the missing part; it prints the one line that fixes it.

VS Code. Download it from code.visualstudio.com. Then in VS Code press Cmd Shift P, type `shell command`, choose "Install 'code' command in PATH". After that `code .` in the terminal opens the current folder. Do not edit these files in TextEdit; it saves rich text, and the tools cannot read that.

---

2. How do you install it in one line?

```
curl -fsSL https://nosey-dewdrop.github.io/dewfpga/install | bash
```

3 min 37 s to 4 min 17 s over three runs on the tested machine, plus about 4 minutes for the yosys-slang step added on 27 September (measured on its own: 70 s of clone, 2 min 53 s of compiling), 1.4 GB downloaded, about 0.3 GB kept on disk (237 MB measured on 7 October; the optional agent SDK adds 51 MB), about 1 GB while it builds, the installer asks for 2 GB free; on slow wifi it takes longer, and it prints progress the whole time. It never asks for a password. It puts the `dewfpga` command into `~/.dewfpga` (its sha256 is compared with `nosey-dewdrop.github.io/dewfpga/dewfpga.tgz.sha256`, so a broken download stops here), links it into Homebrew's bin, installs Yosys, openFPGALoader, Icarus Verilog and the build tools of section 3.1 through Homebrew, and builds nextpnr-xilinx, prjxray and the yosys-slang plugin (section 3.6) into `~/fpga` at the three commits in section 9. Nothing touches your system Python. The last lines it prints:

```
all good.

Total: 217 s. Log: /Users/you/fpga/install.log
Next:  dewfpga new blink && cd blink && dewfpga flash
```

To read the script before running it:

```
curl -fsSL https://nosey-dewdrop.github.io/dewfpga/install -o install.sh
code install.sh      # read it, then:
bash install.sh
```

Check that every piece is there:

```
dewfpga check
```

```
dewfpga check  (FPGA_HOME=/Users/you/fpga)
  ✓ yosys            /opt/homebrew/bin/yosys
  ✓ yosys-slang      /Users/you/fpga/yosys-slang/build/slang.so
  ✓ iverilog         /opt/homebrew/bin/iverilog
  ✓ openFPGALoader   /opt/homebrew/bin/openFPGALoader
  ✓ nextpnr-xilinx   /Users/you/fpga/nextpnr-xilinx/build/nextpnr-xilinx
  ✓ xc7frames2bit    /Users/you/fpga/prjxray/build/tools/xc7frames2bit
  ✓ fasm2frames      /Users/you/fpga/prjxray/utils/fasm2frames.py
  ✓ chipdb           /Users/you/fpga/chipdb/xc7a35t.bin
  ✓ venv             /Users/you/fpga/venv/bin/python
  ✓ python pkgs      fasm prjxray pyyaml textx simplejson intervaltree
all good.
```

A red `✗` names the missing piece: run the curl line again (it is the same as `dewfpga install`), it skips what is done. `dewfpga: command not found` has two causes: the Homebrew "Next steps" lines from section 1 were not pasted (the installer says so, `Homebrew is installed (...) but not on your PATH`, and prints the line to paste), or the installer stopped before its last line; scroll up to the first `ERROR:` line and look it up on the errors page.

Now plug the Basys3 into the Mac with the USB cable (the PROG port, next to the power switch) and flip the power switch on. Make the example project and flash it:

```
dewfpga new blink    # prints: blink/ ready:  cd blink && dewfpga flash    (VS Code: ⌘
⇧B)
cd blink
dewfpga sim          # simulation
dewfpga bit          # .sv -> .bit
dewfpga flash        # .bit -> the board; runs bit first when the .bit is missing or o
lder than the .sv
```

What each one prints when it works. `dewfpga sim`:

```
VCD info: dumpfile blink.vcd opened for output.
PASS: 3 checks (see the waveform for the toggle)
blink_tb.sv:35: $finish called at 200000 (1ps)
```

`dewfpga bit`, three lines: the pin check, place and route, the bitstream. The full place-and-route log is in `blink.log`.

```
xdc ok: 33 ports, all mapped.
pnr ok: 73 LUT, 27 FF, 278.71 MHz (PASS at 100.00 MHz)   (full log: blink.log)
blink.bit  2.2 MB
```

`dewfpga flash`:

```
Jtag frequency : requested 6.00MHz    -> real 6.00MHz
Open file DONE
Parse file DONE
load program
Load SRAM: [==================================================] 100.00%

Done
Shift IR 35
ir: 1 isc_done 1 isc_ena 0 init 1 done 1
```

`done 1` means the chip accepted the design. Flip switch 0 up (SW0, the rightmost of the 16 switches): LED 15 (LD15, leftmost) lights and LED 0 (LD0, rightmost, just above SW0) blinks once a second. The design lives in SRAM, so a power cycle clears it; flash again.

If it prints `ERROR: board not found. Is the USB (PROG port) plugged in and the power switch ON?` instead, the Mac does not see the board: power switch, PROG port, a charge-only cable, the adapter, or the JP2 jumper. The board not found page goes through them in order.

The rules `dewfpga` follows in a folder:

- Every `.sv` and `.v` in the folder is handed to the tools, so submodules can live in their own files, and a file can be named anything: `lab5.sv` may hold `module top_design`. A file that holds no module (a package, an interface, or typedefs at file scope, alone in their file) is read before the files that hold a module, each after the files that declare a name it uses (a package before the files that name it, a typedef file before the interface that uses the type; alphabetical order would hand `iface.sv` before `types.sv` and iverilog would stop at the first `nib_t`), and the files

are one compilation unit for both readers, as Vivado reads the files of one library (UG901 Ch.10, Compilation Units). A file that another file `includes` is read through the include only (handing it too would declare the package twice). A testbench has no ports and instantiates the design. `$finish` or `$stop` in a design module is an error; keep them in the testbench.

- In a flat folder, hierarchy and a unique matching `.xdc` select the top. If several modules qualify, name one: `dewfpga bit lab4` (module or source filename). A module's own filename alone does not select it. `dewfpga tops` lists candidates without changing files.
- Inside a Vivado project, the `.xpr` chooses the enabled design, constraint and simulation sets and their top modules. A command inside its `.srcs` subfolders runs at the project root; outputs go beside the `.xpr`. Without an `.xpr`, a `.srcs` tree with `sources_1`, `constrs_1` and `sim_1` is supported. Missing design files stop the build; a problem confined to the simulation set stops `sim`. Multiple projects or ambiguous active sets produce an error. `.v` files retain Verilog-2005 keywords, and local include directories are passed to both readers. IP Catalog and Block Design generation are not supported.
- The pin file is `<top>.xdc`, or the only `.xdc` in the folder. Pins in it that the design does not use are ignored, so a fully uncommented `Basys3_Master.xdc` is fine.
- `dewfpga sim` needs a testbench that instantiates the top; `<top>_tb.sv` is the usual name but any name works. `bit` and `flash` do not need one. A testbench that never reaches `$finish` is stopped after 120 s with an error; `SIM_TIMEOUT=600 dewfpga sim` waits longer. `$stop` ends the run like `$finish`. `flash` builds the `.bit` first if it is missing or older than the sources, so `dewfpga flash` alone is enough.
- Code that Vivado lets through and Yosys refuses: the course's `SevSeg_4digit.sv` port line (`output [6:0]seg, logic dp`) gets its direction written in by `bit`, with a note naming the line. Some other cases stop with the line and the fix, such as an instance without a name (`clk_div(clk, out);`) or a latch in an `always_comb`. Others stop with no line, or with advice that does not fit, and some build a bitstream that does not do what the code says, with no error. Section 7 lists what the course's files and old student repos hit; section 8 links the full measurement.
- `dewfpga bit` writes no bitstream when timing is not met, and synthesis failures remove the previous `.bit`. A failure during XDC validation can still leave an old `.bit` on disk; a file being present does not prove that the latest build succeeded. Without a `create_clock` in the XDC every clock is checked at 100 MHz, a divided one too, which Vivado does not time; a multiplier that Yosys puts in a DSP48E1 with a register inside is not timed at all (nextpnr-xilinx). When nothing changed, `bit` prints one line saying so. `$display` in synthesizable code is dropped, as Vivado does. `dewfpga sim` exits with an error when the testbench prints `$error` or `$fatal`, so a red run is visible.
- `bit` leaves its working files next to the sources (`.json`, `.fasm`, `.frames`, `.log`, `_routed.json`), `sim` leaves `<top>_sim`, `<top>_sim.out` and `<top>.vcd`, `flash` leaves `<top>_flash.out`. `dewfpga clean` removes all of them and the `.bit`; the `.sv` and `.xdc` stay. Run it before you hand a lab in.

```
dewfpga install                     install the toolchain (safe to re-run)
dewfpga check                       is every piece in place
dewfpga sim|bit|flash|clean [top]   build a flat folder or a Vivado project
dewfpga tops                        list top candidates without writing files
dewfpga vscode [--remove]           set up or remove the editor integration
dewfpga new <dir>                   blink example with a VS Code task (Cmd Shift B = f
lash)
dewfpga uninstall                   remove what install built in ~/fpga, the CLI and i
ts link;
                                    your files in ~/fpga and the brew packages stay
dewfpga --version
```

Continue with section 5.

---

3. How do you build the chain by hand?

These are the commands the installer runs, at the same pinned commits. Vivado does five jobs for a lab; six free tools do them here, two of them for the bitstream:

```
your_design.sv
  |-> Icarus Verilog   -> simulation
  |-> Yosys            -> synthesis (reads SystemVerilog through the yosys-slang plugi
n)
  |-> nextpnr-xilinx   -> place and route
  |-> fasm2frames      -> configuration frames
  |-> xc7frames2bit    -> your_design.bit
  `-> openFPGALoader   -> the board, over USB
```

Three come from Homebrew, three you compile (nextpnr-xilinx, prjxray, and the yosys-slang plugin that Yosys loads to read SystemVerilog), and fasm2frames is a Python script inside the prjxray checkout. nextpnr also needs a chip database, generated once by another Python script. Everything lands in `~/fpga`: about 0.3 GB with the installer (237 MB measured on 7 October; it deletes the clones and build trees after each build; the optional agent SDK adds 51 MB), 1.76 GB with the full clones below until section 3.7 removes what the builds no longer need. The Makefile in section 4 expects exactly this layout: `~/fpga/venv`, `~/fpga/nextpnr-xilinx/build`, `~/fpga/chipdb/xc7a35t.bin`, `~/fpga/prjxray`, `~/fpga/yosys-slang/build/slang.so`.

3.1 Homebrew packages

```
brew install yosys openfpgaloader icarus-verilog cmake ninja eigen pkg-config python@
3.14
```

The first three are tools from the diagram; the rest are what the two builds below need. A few hundred MB. Check:

```
yosys -V                   # Yosys 0.69+post
openFPGALoader --Version   # v1.1.1
iverilog -V | head -1      # Icarus Verilog version 13.0 (stable)
```

Newer numbers are expected; only the ones in section 9 were tested.

3.2 Python venv

fasm2frames (3.5) and the chip database generator (3.4) are Python scripts. Homebrew's Python refuses a plain `pip install` (externally-managed-environment), so they get a private one.

```
mkdir -p ~/fpga
"$(brew --prefix)/opt/python@3.14/bin/python3.14" -m venv ~/fpga/venv
~/fpga/venv/bin/pip install fasm==0.0.2.post88 pyyaml==6.0.3 textx==4.4.0 simplejson==
4.1.2 intervaltree==3.2.1 numpy==2.5.3 pyjson5==2.0.1
```

Check (prints nothing when it works):

```
~/fpga/venv/bin/python -c "import fasm, yaml, textx, simplejson, intervaltree"
```

3.3 nextpnr-xilinx

Place and route. Homebrew has no macOS build, so you compile it. The clone is most of the download; the compile is about 1.5 minutes on this machine.

```
cd ~/fpga
git clone --recursive https://github.com/openXC7/nextpnr-xilinx.git
cd nextpnr-xilinx
git checkout 3fd78784c7788f93f276358edf5477221cc6c179
git submodule update --init --recursive
cmake -B build -G Ninja \
  -DARCH=xilinx -DCMAKE_BUILD_TYPE=Release \
  -DUSE_OPENMP=OFF -DBUILD_GUI=OFF -DBUILD_PYTHON=OFF -DBUILD_TESTS=OFF
ninja -C build -j3 nextpnr-xilinx bbasm
```

`-DUSE_OPENMP=OFF` is required: Apple's compiler has no OpenMP (fopenmp). `-j3` fits 8 GB of RAM; with 16 GB or more use `-j$(sysctl -n hw.ncpu)`. Check:

```
~/fpga/nextpnr-xilinx/build/nextpnr-xilinx --version
# "nextpnr-xilinx" -- Next Generation Place and Route (Version 0.9.6)
```

3.4 Chip database

nextpnr needs a description of the XC7A35T. It is not in the download; you generate it once (no-chipdb). About 35 seconds and 860 MB of RAM while it runs.

```
mkdir -p ~/fpga/chipdb
cd ~/fpga/nextpnr-xilinx
~/fpga/venv/bin/python xilinx/python/bbaexport.py \
  --xray xilinx/external/prjxray-db/artix7 \
  --metadata xilinx/external/nextpnr-xilinx-meta/artix7 \
  --device xc7a35tcpg236-1 \
  --constids xilinx/constids.inc \
  --bba ~/fpga/chipdb/xc7a35t.bba
build/bbasm -l ~/fpga/chipdb/xc7a35t.bba ~/fpga/chipdb/xc7a35t.bin
rm ~/fpga/chipdb/xc7a35t.bba
```

Check:

```
ls -lh ~/fpga/chipdb/xc7a35t.bin
# about 90 MB
```

Another 7-series board would need its own `--device`. Untested.

3.5 prjxray

Turns nextpnr's text output into the binary `.bit`: one Python script, one compiled tool. About 2 minutes.

```
cd ~/fpga
git clone --recursive https://github.com/f4pga/prjxray.git
cd prjxray
git checkout c9f02d85760423254258246 47ab5555b1bc77833
git submodule update --init --recursive
~/fpga/venv/bin/pip install --no-deps -e .
cmake -B build -DCMAKE_BUILD_TYPE=Release -DCMAKE_POLICY_VERSION_MINIMUM=3.5 -DPRJXRAY
_BUILD_TESTING=OFF
cmake --build build --target xc7frames2bit -j3
```

`--recursive` is required (yaml-cpp-not-built). `-DCMAKE_POLICY_VERSION_MINIMUM=3.5` is required (cmake-policy-version-minimum). prjxray's own README says to install Vivado 2017.2; that is for mapping a new chip, and ours is mapped. Ignore it. Check:

```
ls ~/fpga/prjxray/build/tools/xc7frames2bit ~/fpga/prjxray/utils/fasm2frames.py
```

Both paths print back. The full check is `make check` in section 4, one line per tool.

3.6 yosys-slang

The SystemVerilog reader. Homebrew's Yosys 0.69 has no `read_slang` (`yosys -p 'help read_slang'` prints `No such command or cell type: read_slang`), so the plugin is compiled against it. About 4 minutes: the clone with its slang submodule, then 2 min 53 s of compiling with `-j2`.

```
cd ~/fpga
git clone --recursive https://github.com/povik/yosys-slang.git
cd yosys-slang
git checkout 96767863835f3c862cea9c63052b2ce9d0b55884
git submodule update --init --recursive
cmake -B build . -DCMAKE_BUILD_TYPE=Release -DBUILD_AS_PLUGIN=ON \
  -DYOSYS_CONFIG="$(brew --prefix)/bin/yosys-config"
make -C build -j2
```

Run `cmake` from inside the checkout, as written (`.` is the source): slang's own cmake asks `git remote get-url origin` in the current directory for the GitHub address of a header it downloads, and from any other directory the configure stops with `Failed to clone repository: 'MikePopoloski/regex.git'`. The last lines of the build:

```
[100%] Linking CXX shared library ../slang.so
[100%] Built target yosys-slang
```

Check that Yosys loads it:

```
yosys -m ~/fpga/yosys-slang/build/slang.so -p 'help read_slang' | grep 'read_slang \['
#     read_slang [options] [filename]
```

The plugin is tied to the Yosys it was built against: after `brew upgrade yosys`, build it again (`rm -rf build`, then the `cmake` and `make` lines). The installer does this by itself: it keeps the `yosys -V` line it built for in `build/.dewfpga-yosys`, compiles the new plugin into `build.new/` and replaces `build/` only once Yosys loads the new `slang.so`, so a build that fails (no disk, no network) leaves the plugin you have in place. A `slang.so` built by hand with the lines above has no stamp; the installer loads it with `yosys -m`, and if `read_slang` is there it writes the stamp instead of building again. `dewfpga check` loads the plugin the same way each time it runs: a plugin that is there but no longer loads prints `does not load in Yosys 0.69+post (built for another yosys? dewfpga install rebuilds it)`, a missing one prints `MISSING (the SystemVerilog reader: dewfpga install builds it, about 4 min)`, and either way the check ends with `something is missing -> dewfpga install` and exit 1. Without the plugin `make bit` still runs, on Yosys' own reader alone.

3.7 What can go after the builds?

The chain that runs is 237 MB (measured with `du` on 7 October 2026); the rest of the 1.76 GB is clones and build trees. The installer deletes them after each build. By hand, keep these and delete the rest of the four checkouts (every `.git`, the `build/` trees around the files below, the other devices under `prjxray-db`):

- `nextpnr-xilinx/build/nextpnr-xilinx` (3 MB) and `chipdb/` (89 MB)
- `nextpnr-xilinx/xilinx/external/prjxray-db/artix7/` without `xc7a100t`, `xc7a200t`, `gridinfo` and `harness` (83 MB): fasm2frames reads the xc7a35t part from it (`XRAYDB` in the Makefile)

- `prjxray/utils/`, `prjxray/prjxray/` and `prjxray/build/tools/xc7frames2bit` (2 MB): fasm2frames, the package the venv imports, the bit writer
- `yosys-slang/build/slang.so` (9 MB)
- `venv/` (52 MB)

The Makefile of section 4 points at nothing else, so the hand-built chain lands at the same size.

---

4. What is in the project folder?

A project is one folder. Download the files, do not type them:

```
mkdir -p ~/cs223/blink && cd ~/cs223/blink
curl -L -o blink.zip https://nosey-dewdrop.github.io/dewfpga/blink.zip && unzip -q bli
nk.zip && rm blink.zip && ls -A
# .vscode  Makefile  blink.sv  blink.xdc  blink_tb.sv  check_xdc.py
```

file	what it is
`blink.sv`	the design: LED 0 blinks once a second while switch 0 is on, LED 15 follows switch 0
`blink_tb.sv`	the testbench: three checks, prints `PASS` or `FAIL`
`blink.xdc`	Digilent's Basys3 master XDC with the 33 pin lines for `clk`, `sw` and `led` uncommented. Vivado reads the same file
`check_xdc.py`	compares the design's ports with the XDC before place and route, because nextpnr's own error names the wrong port
`Makefile`	runs the six tools in order. `--seed 1` makes every build byte-identical
`.vscode/tasks.json`	Cmd Shift B in VS Code runs `make flash`

`dewfpga new blink` gives the same `blink.sv`, `blink_tb.sv` and `blink.xdc` without the Makefile, and its `tasks.json` says `dewfpga flash` instead.

If you copied the Makefile from somewhere else and `make` says `missing separator`, the tabs became spaces: missing-separator.

Plug the board in over USB, switch it on, then from inside the folder:

```
make check
```

```
yosys            /opt/homebrew/bin/yosys
iverilog         /opt/homebrew/bin/iverilog
openFPGALoader   /opt/homebrew/bin/openFPGALoader
nextpnr-xilinx   /Users/you/fpga/nextpnr-xilinx/build/nextpnr-xilinx
xc7frames2bit    /Users/you/fpga/prjxray/build/tools/xc7frames2bit
chipdb           /Users/you/fpga/chipdb/xc7a35t.bin
fasm2frames      /Users/you/fpga/prjxray/utils/fasm2frames.py
venv             /Users/you/fpga/venv/bin/python
```

Eight lines, each ending in a path, and no –MISSING– anywhere: done. A line such as `chipdb –MISSING–` names the step of section 3 to redo (3.4 here). Then:

```
make sim       # PASS: 3 checks
make bit       # xdc ok / pnr ok / blink.bit, under five seconds
make flash     # Load SRAM ... done 1
```

The outputs are the same three blocks shown in section 2. `make sim` also writes `blink.vcd`. No waveform viewer is installed by this guide; to look at the waveform, open the browser testbench and paste the same `blink.sv` and `blink_tb.sv` there, and it draws it.

---

5. How do you use it for your own lab?

The pin file. The course hands out `Basys3_Master.xdc` with every line commented out. If you do not have it, this puts a copy into the current folder:

```
curl -fsSL -o Basys3_Master.xdc https://nosey-dewdrop.github.io/dewfpga/templates/Basys3_Master.xdc
```

Copy it into your lab folder and delete the leading `#` on the `set_property` lines for the pins your module uses. Uncommenting all of them is fine too: pins the design does not use are ignored. The port names in your top module must match the names in that file: `clk`, `sw[0]`, `led[0]`, `seg[0]`, `an[0]`, `btnC`. If your module says `clock`, change the name inside `[get_ports ...]` on that line. The `xdc` check that runs before place and route lists every port that has no pin and says what to rename.

A design without a clock (`assign led = sw;`) prints `Warning: No clocks found in design` and `no clocked paths, timing not applicable`; that is correct, not a problem. With the whole file uncommented and no `clk` port, `create_clock: target [get_ports clk] matched nothing` is the same thing said by nextpnr.

With `dewfpga`: put the `.sv` files and the `.xdc` in one folder, `cd` into it, `dewfpga flash`. With `lab4.sv` and `Basys3_Master.xdc` in the folder that is all. With several `.sv` files, name the top: `dewfpga flash lab4`.

With the Makefile: copy `Makefile` and `check_xdc.py` from section 4 into the lab folder and change the first three lines.

```
TOP  := traffic_light
SRCS := traffic_light.sv debounce.sv
XDC  := Basys3_Master.xdc
```

`TOP` is the top module's name, `SRCS` every `.sv` in the design, `XDC` the pin file.

Either way, simulation looks for a testbench (with the Makefile: `<top>_tb.sv`); the bitstream does not need one. The course's `SevSeg_4digit.sv` builds as it is when it is source code (`bit` supplies one port direction in a temporary compiler copy); the course also hands it out in netlist form, which the CLI refuses for now. Section 7 says what was changed and why.

---

6. How do you set up VS Code?

Run `dewfpga vscode` to install the companion extension and user tasks in supported local VS Code profiles. Existing JSONC comments, unrelated tasks and shortcuts are preserved. `dewfpga vscode --remove` removes only unchanged entries recorded as dewfpga's; user edits stay. A missing editor or optional setup failure does not prevent terminal use.

Open the Command Palette and use **dewfpga: choose the top module**. The chooser reads `dewfpga tops`, remembers the selection per project, and offers it again on the next build. The tasks use absolute file locations so compiler diagnostics can appear in Problems when the project lives under Vivado's `.srcs` tree. `dewfpga new` also includes project tasks; the Makefile-only download retains its `make` tasks.

For linting, install `mshr-h.veriloghdl`, then explicitly run **dewfpga: use the dewfpga iverilog lint shim**. It changes Icarus's constant-select “sorry” diagnostic to a warning; syntax errors stay errors. **dewfpga: stop using the dewfpga iverilog lint shim** restores the previous setting if it has not been edited since setup. Workspace overrides take precedence. After a successful simulation, the companion can open a new or changed VCD in the project root. An unchanged old waveform is not opened. Nested and absolute dumpfile locations are not discovered automatically.

The command and filesystem behavior is tested in isolated profiles. Real VS Code GUI acceptance remains a separate manual check.

---

7. Which errors does the CLI stop with, and what do they mean?

When `sim`, `bit` or `flash` stops, it prints one message in one format:

```
blink.sv:12: ERROR [code]: what is wrong, in one line
  Fix: what to change
  https://nosey-dewdrop.github.io/dewfpga/errors/code/
```

`blink.sv:12` is the line in your file, not in a copy the tool made. `code` names the entry in the error catalog; the page behind the address holds what you saw, why it happens, the fix and an example line. Warnings and notes carry the same code in the same brackets, without the stop. The catalog, every code on one page: Which error did you get? It is generated from one file, `docs/errors.md`, by `docs/errors-build.sh`.

7.1 Which course file breaks?

Two. One is fixed for you: `SevSeg_4digit.sv` (also handed out as `SevenSegmentDisplay.sv`), as source code, has a port line that Vivado and Icarus accept and Yosys does not (port-neither-input-nor-output). The other is not fixed yet: the ready modules the course hands out in netlist form (LUT primitives, `\<const0>`, `(* keep_hierarchy *)`), `SevSeg_4digit.sv` among them, build in Vivado, but the CLI takes them for a netlist Vivado left behind and stops `sim` and `bit` with "Delete it from this folder". 5 of the 89 folders measured from old student repos hold such a module: 4 are one repo's copy of the course's handout folder (its example projects and ready modules), 1 is a student's project that copied the modules in.

```
output [6:0]seg, logic dp,     // dp inherits `output` from the previous port, per the
standard
```

Yosys does not implement that inheritance. `dewfpga bit` supplies the direction in a temporary compiler copy before synthesis and tells you so. Your saved file is unchanged:

```
SevSeg_4digit.sv:4: note [port-neither-input-nor-output]: wrote the port direction in:
output [6:0]seg, output logic dp,   (in the copy the tools read, your file stays as yo
u wrote it; the standard and Yosys need it; Vivado accepts both)
```

The original file is preserved, so it remains available to other tools exactly as you wrote it. The fix misses the line when a comment sits between `seg,` and `logic dp` (at the end of the `seg,` line, on a line of its own, or a `/* */` between the two), or when a comment in the port list holds a `;`; then Yosys stops with `Module port `\dp' is neither input nor output.` and no fix, so move the comment out of the way. More things Vivado lets through and Yosys refuses came up in old student repos. One more port-line shape is fixed the same way, by `bit`; the rest stop the build and name the line, except where the item says otherwise:

- An instance without a name, `trans_3s_clock(clk, reset, out);`. The standard requires a name: `trans_3s_clock u1(clk, reset, out);`.
- A latch in an `always_comb` block: a signal not assigned on every path (an `if` without `else`, a `case` without `default`). Vivado builds a latch and warns; Yosys refuses. Give the signal a default value at the top of the block.
- A port list where a later port only has a range, `input logic [7:0] d, [2:0] s`: `bit` writes the direction in, like the seven-segment line above; `sim` does not. When such an output is written in an `always` block (`output logic [6:0] seg, [3:0] an`), `sim` stops with "'an' is not a valid l-value" until a `bit` has rewritten the line.

- A reset branch that loads a signal, not a constant, in an `always_ff` with `posedge reset`: the bitstream is built with Yosys warnings only, and while the reset is held the board does not do what `sim` shows. A register written on both clock edges stops the build, with no line number.
- An asynchronous reset ORed with a synchronous clear, `always_ff @(posedge clk, posedge reset) if (reset || clear)`: the build stops with "Multiple edge sensitive events found for this signal!", no line number, and advice about two always blocks that does not fit one.
- A statement after the reset's `if/else` in an `always_ff` with an asynchronous reset: the build stops with "Async reset ... yields non-constant value", no line number, and advice about the reset branch.
- A file Vivado wrote after synthesis (`\<const0>`, `keep_hierarchy`) copied in as if it were source: the build stops, names the file and says to delete it. The same check also stops the course's netlist-form modules above, and a module with UG901's own `(* keep_hierarchy = "yes" *)` attribute.

What else was tried: four modules from old student repos, together in one design. An FSM with `typedef enum`, a 16x8 RAM, a debouncer and the seven-segment driver above: 136 LUTs, 48 flip-flops, timing passes at 154.70 MHz against the 100 MHz clock, bitstream in 4.5 seconds. Language features in those files that both Icarus (`sim`) and Yosys (`bit`) accepted: `parameter`, `generate`, `struct packed`, `$clog2`, `unique case`, packed arrays, `interface` with `modport` (Icarus 13 accepts one only while no module port is typed with it; a port like `sum_if.prod bus` fails `sim` and builds in `bit`). A submodule file named differently from its module, Turkish characters in comments, and spaces in the folder path went through too. A file saved as "UTF-8 with BOM" stops the build, with the command that removes the mark. That is four files, not the whole course.

---

8. What can it not do?

This covers the part of Vivado that CS223 uses, not all of it yet, and nothing beyond it.

- **Nothing from Vivado's IP Catalog.** No Block Design, MicroBlaze, AXI, Clocking Wizard, or the BRAM, VGA and UART cores. 35 old student repos with 240 source files were searched for those; none use any of it. (Searched, not synthesized.)
- **No GUI.** No waveform viewer, schematic or in-chip debugger. `sim` writes a `.vcd`; the browser testbench draws the waveform for the same files.
- **SystemVerilog that Vivado compiles and this chain does not, yet.** `test/sv` in the repository holds one probe for each construct measured so far, each run through `sim`, `bit` and a simulation of the synthesized netlist, next to what Vivado does and the source for it (`expect.tsv`). It is not every construct: each round of testing has found more. On 26 September, 82 of the 111 constructs Vivado supports failed a stage: 58 stop before a bitstream (Yosys refuses them, or the CLI, place and route or its timing check does), 12 fail only in `sim` (Icarus 13 refuses them or has no model), 2 (an MMCM, and a memory marked as a block RAM) build but have no model to check them against, and 10 build a bitstream that does not do what the code says, with no error: a hierarchical name such as `u_fsm.state`, `$isunknown`, the members of an interface used from the module that instantiates it (`bus.a`, with or without ports, or in an array of interfaces: 3 probes), a package in its own file, reading

an `inout` pin the design sets to a constant `z`, a design module that calls `$finish`, a packed 2-D `localparam` whose type is a `typedef` (the table comes out as zeros), and two modules nothing instantiates where one is named like its own file (the CLI builds that one, not the top your project names).

- **Two parsers.** Icarus reads your code for simulation, Yosys for synthesis, and they do not support the same SystemVerilog. Code that passes `sim` can fail in `bit`; section 7 lists what the course's files and old student repos hit, and `expect.tsv` above every construct measured.
- **The timing number is nextpnr's estimate.** Vivado's report is the official one. In the two designs above the margin was 1.5x (four modules) and 2.8x (blink) over 100 MHz.
- **Unofficial tools.** AMD does not make or support them; prjxray worked out the bitstream format without AMD's documentation. If a lab is graded in Vivado, open it in Vivado once before the demo.
- **Memories become distributed RAM** (`–nobram`). The 16x8 RAM in section 7 mapped to four RAM32M cells. A large memory such as a VGA framebuffer would not fit this way; big multipliers and DSP blocks are untested. The one exception is a memory the code asks to be a block RAM, `(* ram_style = "block" *)` or `(* rom_style = "block" *)`: Yosys builds a RAMB18E1 then, which the timing check does not time and the test suite cannot simulate (`97_ram_style_block`).
- **One board, one platform.** Basys3 (XC7A35T), Apple Silicon. Intel Mac, Linux and other boards are untested and the installer refuses them.
- **Confirmed on the board** (19 and 20 September): five designs: `blink`, switches to LEDs, the Lab 2 adder/subtractor, the button counter on the display and the traffic-light FSM with sensor hold (the last three are the simulator's examples). The four-module design in section 7 went as far as a bitstream and was not loaded.
- **`` `ifdef SIM ``.** The browser simulator defines `SIM`; the command line does not. A design can use it to shorten a clock divider for the browser and keep the real value for the board, as `blink.sv` does.

---

9. Which versions were tested?

The installer pins nextpnr-xilinx, prjxray and yosys-slang to these commits; section 3 checks out the same ones. Homebrew packages cannot be pinned; the versions below are what was tested.

```
Yosys              0.69+post (git 143eb14f)
nextpnr-xilinx     0.9.6, openXC7 commit 3fd7878
prjxray            commit c9f02d8; prjxray-db 0.9.1-30-g1768fb3 (artix7)
yosys-slang        commit 9676786 (slang 11.0.0+e222e7d), built against the Yosys abov
e
openFPGALoader     1.1.1
Icarus Verilog     13.0 (stable)
Python             3.14 (Homebrew), venv under ~/fpga

Board              Digilent Basys3, Artix-7 XC7A35T-1CPG236C
Machine            Apple M2, 8 GB RAM, macOS 15 (Darwin 24.2.0)
```
